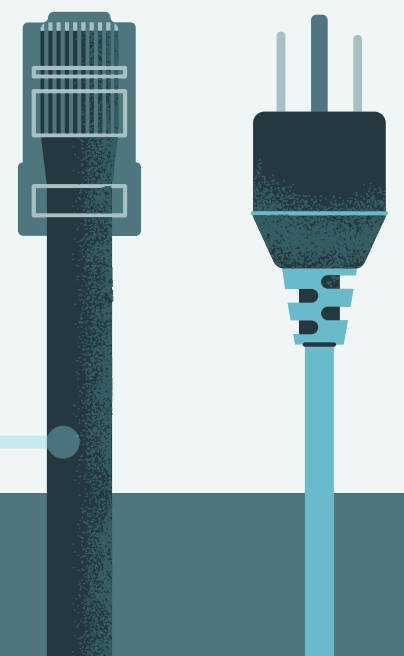
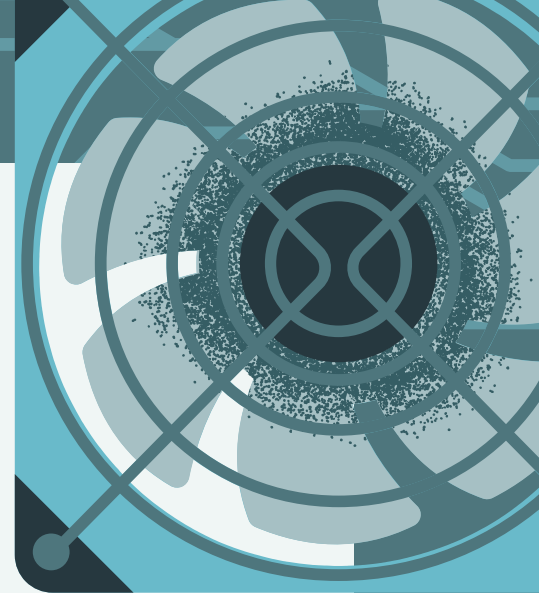


4 de junio de 2026

CONSTRUCCION DE SOFTWARE

- Chavez Miranda Leonardo Gabriel
- Ramos Jacay Dair David
- Eustaquio Avila Luis Gabriel
- Paucar Ventura Luis Antonio
- Chupa Ballesteros Gabriel

SISTEMA DE GESTIÓN ACADÉMICA





AVANCE DEL DISEÑO DE PANTALLAS



LOGIN



Universidad Nacional de Ingeniería

✓ Institución detectada · uni.edu.pe

Correo institucional

✉ Ichavez@uni.edu.pe

Contraseña

🔒 ••••••••

[¿Olvidaste tu contraseña?](#)

Iniciar sesión →

Acceso de demostración

🎓 Alumno

📄 Docente

👤 Administrador

📁 Admin Central

Plataforma multi-institucional **SGA** · app.sgaplatform.com



PANTALLAS ADMINISTRADOR DE PLATAFORMA



PANTALLA DE ADMINISTRADOR DE LA PLATAFORMA

SGA Platform

Consola central

Instituciones

<< Contraer

app.sgaplatform.com

Operador Plataforma Administrador Central

SGA Platform > Instituciones

Instituciones

Tenants activos en la plataforma. Cada institución opera de forma totalmente aislada, con sus propios catálogos, usuarios y datos académicos.

+ Registrar institución



6

Instituciones registradas



5

Activas



16,419

Alumnos en la plataforma



99.97%

Disponibilidad (30 días)

INSTITUCIÓN	DOMINIO	ZONA HORARIA	ALUMNOS	ESTADO	ÚLTIMA ACTIVIDAD
U UNI Universidad Nacional de Ingeniería	uni.edu.pe	America/Lima	1,284	Activo	2026-06-03 09:42
P PUC Pontificia Universidad Católica	pucp.edu.pe	America/Lima	2,640	Activo	2026-06-03 08:15
U UNMSM Universidad Nacional Mayor de San Marcos	unmsm.edu.pe	America/Lima	3,110	Activo	2026-06-02 21:08
U UBA Universidad de Buenos Aires	uba.ar	America/Argentina/Buenos_Aires	4,205	Activo	2026-06-03 07:51



PANTALLAS ADMINISTRADOR ACADÉMICO



PANTALLA DE ADMINISTRADOR ACADÉMICO

U

UNI

Año académico 2026

GENERAL

Panel principal

Períodos académicos

CONFIGURACIÓN

Oferta académica

Políticas académicas

Usuarios

ACADÉMICO

Corrección de notas

Auditoría

Reportes de rendimiento

<< Contraer

2026-I

MATRÍCULA

Dirección Académica

Administrador Académico

DA

UNI > Panel principal

Panel principal

Universidad Nacional de Ingeniería · Facultad de Ingeniería Industrial y de Sistemas

Ver auditoría

Gestionar período

PERÍODO ACTIVO

2026-I

23 mar – 24 jul 2026

Fase: Matrícula

1

Configuración

2

Matrícula

3

Registro de notas

4

Cerrado

1,284

Alumnos matriculados

86

Secciones activas

42

Actas pendientes de cierre

23

Alumnos con condición académica

Avance de matrícula por carrera

Ingeniería de Sistemas

642 / 720 alumnos

Ingeniería Industrial

588 / 664 alumnos

Condiciones académicas

Regular

1261

Riesgo académico

18

PANTALLA DE ADMINISTRADOR ACADÉMICO

U

UNI

Año académico 2026

GENERAL

Panel principal

Períodos académicos

CONFIGURACIÓN

Oferta académica

Políticas académicas

Usuarios

ACADÉMICO

Corrección de notas

Auditoría

Reportes de rendimiento

<<

Contraer

2026-I

REGISTRO DE NOTAS

Dirección Académica

Administrador Académico

DA

UNI > Períodos académicos

Períodos académicos

Cada período recorre cuatro fases de forma unidireccional e irreversible. Solo puede haber un período activo en Matrícula o Registro de notas a la vez.

+ Crear período

2026-I

Registro de notas

Activo

23 mar – 24 jul 2026

✓

Configuración

✓

Matrícula

3

Registro de notas

4

Cerrado

ⓘ

Siguiente fase: Cerrado. El avance es irreversible.

→ Cerrar período

Historial de períodos

PERÍODO	INICIO	FIN	ESTADO	
2026-I	2026-03-23	2026-07-24	Registro de notas	>
2025-II	2025-08-18	2025-12-19	Cerrado	>
2025-I	2025-03-24	2025-07-25	Cerrado	>

PANTALLA DE ADMINISTRADOR ACADÉMICO

U

UNI

Año académico 2026

GENERAL

Panel principal

Períodos académicos

CONFIGURACIÓN

Oferta académica

Políticas académicas

Usuarios

ACADÉMICO

Corrección de notas

Auditoría

Reportes de rendimiento

<<

Contraer

2026-I

CONFIGURACIÓN

Dirección Académica

Administrador Académico

DA

UNI > Configuración > Oferta académica

Oferta académica

Plan de estudios, secciones, docentes y componentes de evaluación del período. Se configura durante la fase de Configuración.

Catálogo de cursos

50 cursos

CICLO 1

CB101

Cálculo Diferencial

4 cr.

CB102

Álgebra Lineal

3 cr.

CB103

Química General

3 cr.

EG101

Redacción y Comunicación

2 cr.

CC101

Introducción a la Computación

3 cr.

BI101

Cálculo Diferencial

4 cr.

BI102

Álgebra Lineal

3 cr.

BI103

Introducción a la Ingeniería Industrial

3 cr.

CB101

Cálculo Diferencial

4 créditos · Obligatorio · Ciclo 1 · Sin prerrequisitos

+ Nueva sección

Sin secciones configuradas

Este curso aún no tiene secciones en el período activo. Crea una sección para asignar docentes, vacantes y componentes.

+ Crear primera sección

PANTALLA DE ADMINISTRADOR ACADÉMICO

U

UNI

Año académico 2026

GENERAL

Panel principal

Períodos académicos

CONFIGURACIÓN

Oferta académica

Políticas académicas

Usuarios

ACADÉMICO

Corrección de notas

Auditoría

Reportes de rendimiento

<<

Contraer

2026-I

CONFIGURACIÓN

Dirección Académica

Administrador Académico

DA

UNI > Configuración > Políticas académicas

Políticas académicas

Reglas paramétricas del período. Todos los límites de créditos, plazos y fórmulas son valores configurables — nunca se codifican en el sistema.

Política de créditos

Límite máximo de créditos según el rango de PPA del alumno.

PPA MÍNIMO	PPA MÁXIMO	CRÉDITOS MÁX.	
0.00	11.99	16	
12.00	13.99	20	
14.00	16.99	22	
17.00	20.00	24	

+ Agregar rango

Política de retiro

Dispersión de matrícula

PANTALLA DE ADMINISTRADOR ACADÉMICO

U

UNI

Año académico 2026

GENERAL

Panel principal

Periodos académicos

CONFIGURACIÓN

Oferta académica

Políticas académicas

Usuarios

ACADÉMICO

Corrección de notas

Auditoría

Reportes de rendimiento

<<

Contraer

2026-I

CONFIGURACIÓN

Dirección Académica

Administrador Académico

DA

UNI > Usuarios

Usuarios

Gestiona las cuentas de alumnos y docentes de la institución. La desactivación conserva el historial académico.

+ Registrar usuario

Estudiantes 10

Docentes 5

Q

Buscar por nombre o código...

USUARIO	CÓDIGO	CORREO	CARRERA	ESTADO	ÚLTIMO ACCESO		
LM Leonardo Chávez Miranda	20240110J	lchavez@uni.edu.pe	Ingeniería de Sistemas	Activo	2026-06-03 09:18		
DJ Dair Ramos Jacay	20232156D	dramos@uni.edu.pe	Ingeniería de Sistemas	Activo	2026-06-03 08:40		
LA Luis Eustaquio Ávila	20240206G	leustaquio@uni.edu.pe	Ingeniería de Sistemas	Activo	2026-06-02 22:05		
LV Luis Paucar Ventura	20240124K	lpaucar@uni.edu.pe	Ingeniería de Sistemas	Activo	2026-06-03 07:12		
GB Gabriel Chupa Ballesteros	20240160G	gchupa@uni.edu.pe	Ingeniería de Sistemas	Activo	2026-06-01 19:30		
AH Andrea Flores Huamán	20231044B	aflores@uni.edu.pe	Ingeniería de Sistemas	Activo	2026-06-03 06:55		
			Ingeniería de				

PANTALLA DE ADMINISTRADOR ACADÉMICO

U

UNI

Año académico 2026

GENERAL

Panel principal

Períodos académicos

CONFIGURACIÓN

Oferta académica

Políticas académicas

Usuarios

ACADÉMICO

Corrección de notas

Auditoría

Reportes de rendimiento

2026-I

REGISTRO DE NOTAS

Dirección Académica

Administrador Académico

DA

UNI > Corrección de notas

Corrección de notas

Registra correcciones sobre notas en actas cerradas. Cada corrección genera un evento inmutable que referencia al registro original y recalcula el PPA.

Leonardo

LM

Leonardo Chávez Miranda

202401103

Período

2025-II

Curso

CC401 · Bases de Datos

Sección

BD-A

Componente

Examen Final

Estado del acta

Cerrado

Valor actual

13

Trazabilidad de correcciones

Registro inmutable de correcciones previas sobre esta nota. Cada entrada conserva el valor anterior.

9

 →

13

Más reciente

Recalificación de examen sustentada por el alumno.

2026-05-20 11:05 · Dirección Académica

Aplicar corrección

Nuevo valor *

Escala 0-20

Justificación *

Describe el motivo de la corrección. Este campo es

<< Contraer

PANTALLA DE ADMINISTRADOR ACADÉMICO

U

UNI

Año académico 2026

GENERAL

Panel principal

Períodos académicos

CONFIGURACIÓN

Oferta académica

Políticas académicas

Usuarios

ACADÉMICO

Corrección de notas

Auditoría

Reportes de rendimiento

2026-I

REGISTRO DE NOTAS

Dirección Académica

Administrador Académico

DA

UNI > Auditoría

Registro de auditoría

Bitácora inmutable de todas las operaciones académicas. Esta vista es de solo lectura — no admite ninguna acción de edición.

Cada evento queda documentado con fecha, usuario responsable, entidad afectada y valores antes/después. Los registros se conservan por un mínimo de 5 años.

01/06/2026

a

03/06/2026

Todas las operaciones

Todos los resultados

Exportar

	FECHA Y HORA	OPERACIÓN	USUARIO	ENTIDAD	RESULTADO
>	2026-06-03 09:42:11	PUBLICAR_COMPONENTE	admin@uni.edu.pe	Componente · CS-A · Examen Parcial	Exitosa
>	2026-06-03 09:18:55	REGISTRAR_MATRICULA	lchavez@uni.edu.pe	Inscripción · CS-A	Exitosa
>	2026-06-03 08:50:02	REGISTRAR_MATRICULA	jvega@uni.edu.pe	Inscripción · CS-B	Rechazada
>	2026-06-02 17:30:41	AVANZAR_PERIODO	admin@uni.edu.pe	Periodo · 2026-I	Exitosa
>	2026-06-02 16:12:08	REGISTRAR_MATRICULA	cmendoza@uni.edu.pe	Inscripción · IO-A	Rechazada

<<

Contraer

PANTALLA DE ADMINISTRADOR ACADÉMICO

U

UNI

Año académico 2026

GENERAL

Panel principal

Períodos académicos

CONFIGURACIÓN

Oferta académica

Políticas académicas

Usuarios

ACADÉMICO

Corrección de notas

Auditoría

Reportes de rendimiento

<<

Contraer

2026-I

REGISTRO DE NOTAS

Dirección Académica

Administrador Académico

DA

UNI > Reportes de rendimiento

Reportes de rendimiento

Indicadores consolidados de la institución. Las métricas de aprobación usan el umbral configurado (11/20).

Exportar PDF

Período

2026-I

Carrera

Ingeniería de Sistemas

Curso

Todos

13.40

Promedio general

77%

Tasa de aprobación

5

Secciones analizadas

146

Alumnos evaluados

Distribución de calificaciones

2

5

7

14

22

9

3

Comparación por sección

CS-A · Construcción de Software

prom. 13.90

CS-B · Construcción de Software

prom. 12.40

SO-A · Sistemas Operativos

prom. 14.60

MN-A · Métodos Numéricos

prom. 11.80

IO-A · Investigación de Operaciones II

prom. 13.20



PANTALLAS DOCENTE



PANTALLA DOCENTE

U

UNI

Año académico 2026

Mis secciones

Registro de calificaciones

Cierre de acta

Rendimiento

2026-I

REGISTRO DE NOTAS

Mag. Juan L. Herencia

Docente

MH

UNI > Mis secciones

Mis secciones

Secciones que tienes asignadas en el período activo. Registra calificaciones por componente y cierra tus actas.

2

Secciones asignadas

61

Alumnos activos

0 / 2

Actas cerradas

2

Componentes publicados

CC502

Coordinador

En progreso

Construcción de Software

Sección CS-A 31 alumnos Lun-Mié 08:00-10:00

Componentes publicados 2 / 4

Examen Parcial

Examen Final

Prácticas Calificadas

Proyecto de Laboratorio

Registrar notas

Cierre de acta

CC503

Coordinador

En progreso

Sistemas Operativos

Sección SO-A 30 alumnos Mar-Jue 10:00-12:00

Componentes publicados 1 / 3

Examen Parcial

Examen Final

Laboratorios

Registrar notas

Cierre de acta

<< Contraer

PANTALLA DOCENTE

U

UNI

Año académico 2026

Mis secciones

Registro de calificaciones

Cierre de acta

Rendimiento

2026-I

REGISTRO DE NOTAS

Mag. Juan L. Herencia

Docente

MH

UNI > Mis secciones > Registro de calificaciones

Registro de calificaciones

Construcción de Software · Sección CS-A

Componente: Examen Final (25%)

Borrador

Escala 0–20 · aprueba ≥ 11

31 alumnos activos

23 / 31 con nota

#	CÓDIGO	ALUMNO	NOTA	ESTADO
1	20220105F	Espinoza Cueva, Diego	—	Pendiente
2	20230118M	Gutiérrez Pari, Micaela	12	Registrada
3	20240131A	Aliaga Bonifaz, Rodrigo	8	Registrada
4	20220144J	Chávez Miranda, Bruno	12	Registrada
5	20230157K	Ramos Jacay, Mariana	—	Pendiente
6	20240170G	Eustaquio Ávila, Leonardo	13	Registrada

<<

Contraer

PANTALLA DOCENTE

U

UNI

Año académico 2026

Mis secciones

Registro de calificaciones

Cierre de acta

Rendimiento

2026-I

CERRADO

Mag. Juan L. Herencia

Docente

MH

UNI > Mis secciones > Cierre de acta

Cierre de acta

Construcción de Software · Sección CS-A · El cierre es irreversible y formaliza las notas finales del curso.

Lista de verificación previa al cierre

Demo: simular componentes publicados

Todos los componentes publicados

2 / 4 componentes en estado PUBLICADO

Todos los alumnos activos calificados

31 / 31 alumnos con nota en cada componente

Periodo habilitado

El período permite el cierre de actas

Vista previa de notas finales

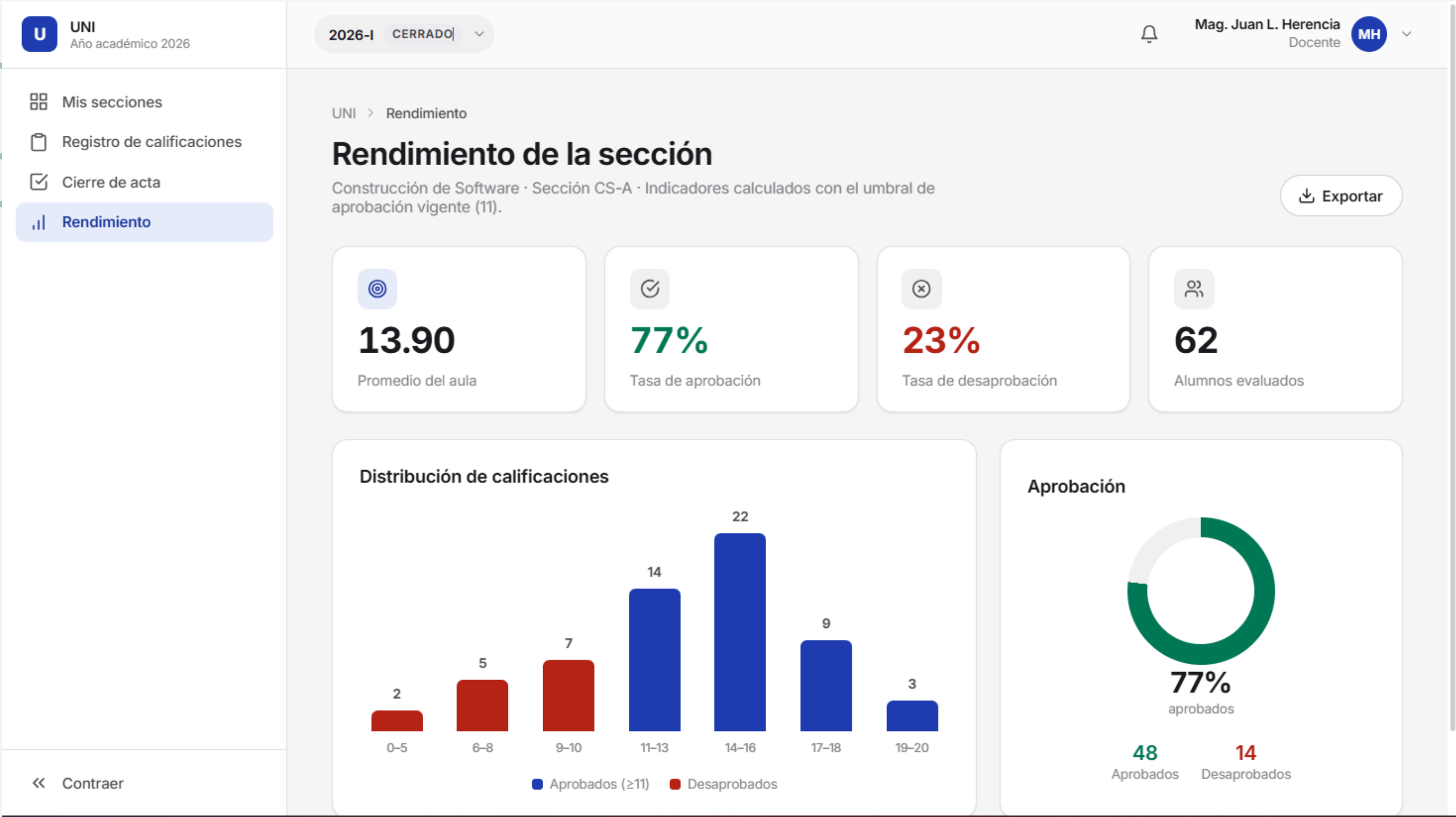
Cálculo según la fórmula configurada: $\Sigma(\text{nota} \times \text{créditos}) / \Sigma(\text{créditos})$

ALUMNO	EXAMEN 25%	EXAMEN 25%	PRÁCTICAS 30%	PROYECTO 20%	FINAL	RESULTADO
Espinoza Cueva, Diego	17	12	10	17	13.65	Aprobado
Gutiérrez Pari, Micaela	19	12	14	16	15.15	Aprobado

<<

Contraer

PANTALLA DOCENTE





PANTALLAS ALUMNO



PANTALLA ALUMNO

U

UNI

Año académico 2026

Inicio

Matrícula

Retiro de cursos

Consulta de notas

Historial académico

Récord de notas

2026-I CERRADO

Leonardo Chávez Miranda

Alumno

LM

UNI > Inicio

Hola, Leonardo

Resumen de tu situación académica en el período activo.

LC

Leonardo Chávez Miranda

20240110J · Ingeniería de Sistemas · Ciclo 5º · Ingreso 2024-I

PPA acumulado

14.62

↑ +0.31

Situación: Regular

4

Cursos inscritos · 14 créditos

14 / 22

Créditos · límite por tu PPA

2

Notas publicadas disponibles

29%

64 / 220 créditos de carrera

Período activo

Matrícula abierta

2026-I

23 mar – 24 jul 2026

Semana 7 de 17

La matrícula cierra en la semana 1. El retiro de cursos está disponible hasta la semana 10.

Accesos rápidos

Ir a matrícula

Inscríbete en la oferta del período

Consultar notas

Avance parcial por componente

Historial académico

Contraer

PANTALLA ALUMNO

U

UNI

Año académico 2026

Inicio

Matrícula

Retiro de cursos

Consulta de notas

Historial académico

Récord de notas

<<

Contraer

2026-I

MATRÍCULA

Leonardo Chávez Miranda

Alumno

LM

UNI > Matrícula

Matrícula 2026-I

Selecciona tus secciones. La oferta se filtra automáticamente según prerequisites cumplidos, vacantes y dispersión de ciclos.

OFERTA DISPONIBLE · CICLO 5-6

CC501

Métodos Numéricos

3 créditos · Ciclo 5

Ya inscrito

MN-A

Dra. R. Menéndez · Vie 08:00–11:00

6 vac.

CC502

Construcción de Software

4 créditos · Ciclo 5

Ya inscrito

CS-A

Mag. J. Herencia · Lun-Mié 08:00–10:00

3 vac.

CS-B

Mag. C. Paredes · Lun-Mié 16:00–18:00

Sin vacantes

CC503

Sistemas Operativos

4 créditos · Ciclo 5

Ya inscrito

SO-A

Mag. J. Herencia · Mar-Jue 10:00–12:00

5 vac.

CC504

Investigación de Operaciones II

3 créditos · Ciclo 5

Ya inscrito

Tu matrícula

Créditos14 / 22 máx.

Límite calculado por tu PPA 14.62 (política de créditos vigente).

Inscripciones confirmadas

Construcción de Software

CS-A · 4 cr.

Sistemas Operativos

SO-A · 4 cr.

Métodos Numéricos

MN-A · 3 cr.

Investigación de Operaciones II

IO-A · 3 cr.

Agrega cursos desde la oferta de la izquierda.

Confirmar matrícula

El sistema valida vacantes, prerequisites, grupos de horario y

PANTALLA ALUMNO

U

UNI

Año académico 2026

Inicio

Matrícula

Retiro de cursos

Consulta de notas

Historial académico

Récord de notas

<<

Contraer

2026-I

MATRÍCULA

▼

🔔

Leonardo Chávez Miranda

Alumno

LM

▼

UNI > Retiro de cursos

Retiro de cursos

Puedes retirar un curso dentro del plazo definido por la política del período. Un retiro queda registrado en tu historial y no afecta tu PPA.

🕒

Plazo de retiro: hasta la **semana 10** del período. Actualmente estás en la **semana 7** — quedan **3 semanas**.

CURSO	SECCIÓN	DOCENTE	CRÉDITOS	ESTADO	
CC502 Construcción de Software	CS-A	Mag. J. Herencia	4	● Activa	Retirar
CC503 Sistemas Operativos	SO-A	Mag. J. Herencia	4	● Activa	Retirar
CC501 Métodos Numéricos	MN-A	Dra. R. Menéndez	3	● Activa	Retirar
CC504 Investigación de Operaciones II	IO-A	Mag. C. Paredes	3	● Activa	Retirar

PANTALLA ALUMNO

U

UNI

Año académico 2026

Inicio

Matrícula

Retiro de cursos

Consulta de notas

Historial académico

Récord de notas

2026-I

REGISTRO DE NOTAS

Leonardo Chávez Miranda

Alumno

LM

UNI > Consulta de notas

Consulta de notas

Avance por componente en tus cursos del período activo. Solo se muestran las notas de los componentes ya publicados por el docente.

CC502

Construcción de Software

Sección CS-A · Lun-Mié 08:00-10:00

Promedio parcial

15.55 / 20

COMPONENTE	PESO	DOCENTE	NOTA	ESTADO
Examen Parcial	25%	Mag. Luis Herencia	15	Publicado
Examen Final	25%	Mag. Luis Herencia	—	Borrador
Prácticas Calificadas	30%	Mag. Paredes Lazo	16	Publicado
Proyecto de Laboratorio	20%	Mag. Quispe Aliaga	—	Borrador

Promedio parcial referencial sobre los componentes publicados (55% del total). No constituye la nota final del curso.

CC503

Sistemas Operativos

Sección SO-A · Mar-Jue 10:00-12:00

Promedio parcial

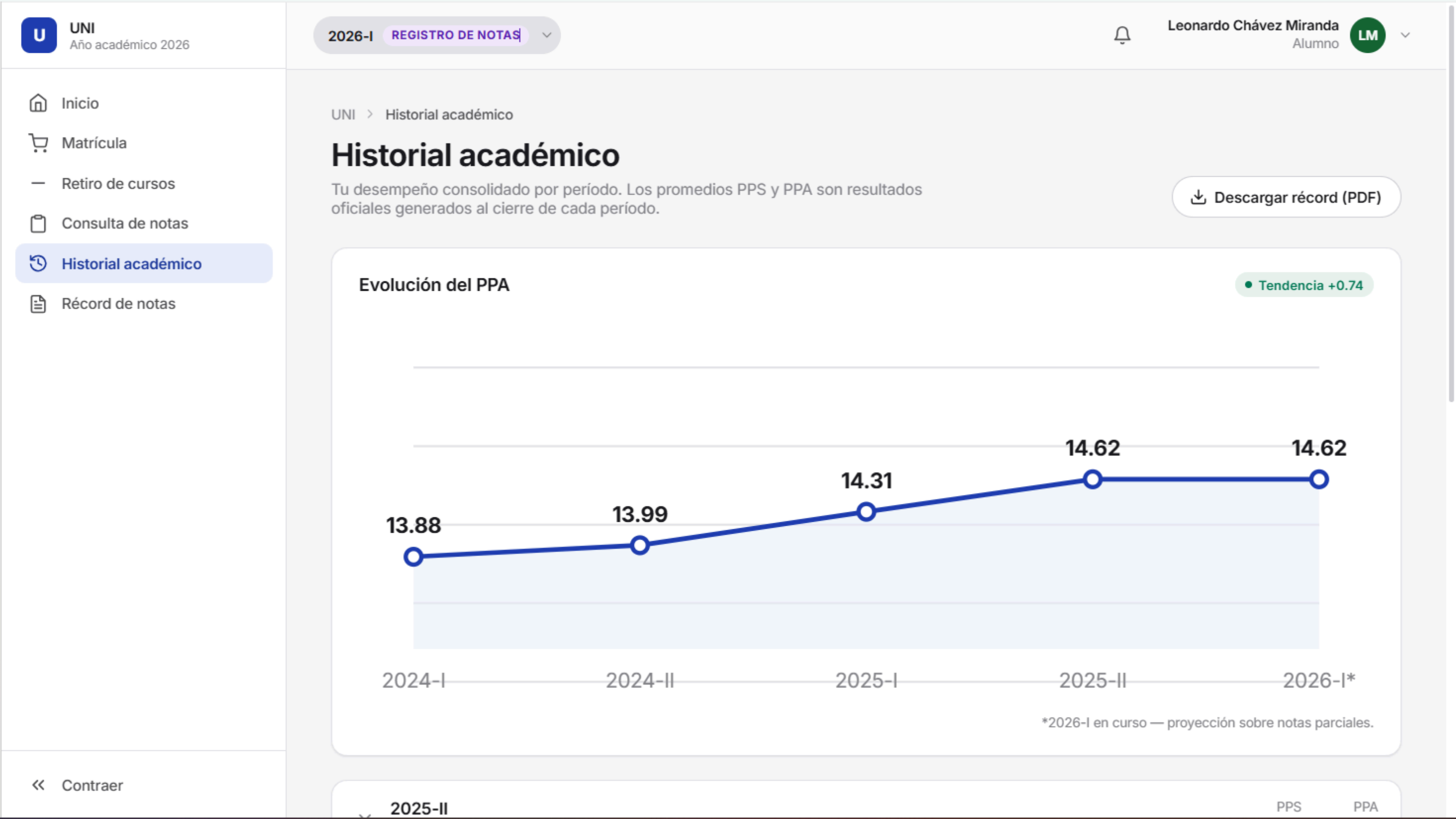
14.00 / 20

COMPONENTE	PESO	DOCENTE	NOTA	ESTADO
------------	------	---------	------	--------

<<

Contraer

PANTALLA ALUMNO



PANTALLA ALUMNO

U

UNI

Año académico 2026

Inicio

Matrícula

Retiro de cursos

Consulta de notas

Historial académico

Récord de notas

<<

Contraer

2026-I

REGISTRO DE NOTAS

Leonardo Chávez Miranda

Alumno

LM

UNI > Récord de notas

Récord de notas oficial

Vista previa del documento oficial. Refleja tu estado académico al momento de la emisión y queda sellado con fecha y hora.

Descargar PDF

U

Universidad Nacional de Ingeniería

Facultad de Ingeniería Industrial y de Sistemas

DOCUMENTO OFICIAL

Récord de Notas

Alumno	Leonardo Chávez Miranda	Código	20240110J
Carrera	Ingeniería de Sistemas	Plan	2018-2
PPA acumulado	14.62	Créditos aprobados	64 / 220

PERÍODO 2024-I

PPS 13.88

CB101 Cálculo Diferencial	4	14	Aprobado
CB102 Álgebra Lineal	3	13	Aprobado
CB103 Química General	3	12	Aprobado
EG101 Redacción y Comunicación	2	16	Aprobado
CC101 Introducción a la Computación	3	15	Aprobado

PERÍODO 2024-II

PPS 14.10



PLAN DE PRUEBAS





PRUEBAS UNITARIAS



AUTENTICACIÓN Y GESTIÓN DE SESIÓN



ID	Caso de prueba	Tipo	Descripción	Resultado esperado
DOM-01	test_validar_dominio_tenant_ok	Unitaria	Nombre de dominio válido (uni-demo)	No lanza excepción
DOM-02	test_validar_dominio_tenant_invalido	Unitaria	Nombres inválidos (Uni_Demo — mayúsculas y guion bajo)	Lanza ValueError
DOM-03	test_validar_escala_evaluacion_ok	Unitaria	Escala válida: mínima=0, máxima=20, aprobatoria=11	No lanza excepción
DOM-04	test_validar_escala_evaluacion_invalida	Unitaria	Escala inválida: aprobatoria=25 supera máxima=20	Lanza ValueError
DOM-05	test_admin_no_puede_crear_admin_central	Unitaria	Rol ADMIN intenta crear usuario ADMIN_CENTRAL	Lanza ValueError de jerarquía
DOM-06	test_cuenta_bloqueada_falso_si_no_hay_fecha	Unitaria	cuenta_bloqueada(None) — sin fecha de bloqueo	Retorna False
DOM-07	test_calcular_bloqueo_por_intentos	Unitaria	calcular_bloqueo(5) — 5 intentos fallidos	Retorna objeto de bloqueo distinto de None

```
1  import pytest
2
3  from app.domain.auth import calcular_bloqueo, cuenta_bloqueada
4  from app.domain.tenant import validar_dominio_tenant
5  from app.domain.users import validar_escal_a_evaluacion, validar_rol_usuario_creacion
6
7
8  def test_validar_dominio_tenant_ok() -> None:
9      validar_dominio_tenant("uni-demo")
10
11
12  def test_validar_dominio_tenant_invalido() -> None:
13      with pytest.raises(ValueError):
14          validar_dominio_tenant("Uni_Demo")
15
16
17  def test_validar_escal_a_evaluacion_ok() -> None:
18      validar_escal_a_evaluacion(minima=0, maxima=20, aprobatoria=11)
19
20
21  def test_validar_escal_a_evaluacion_invalida() -> None:
22      with pytest.raises(ValueError):
23          validar_escal_a_evaluacion(minima=0, maxima=20, aprobatoria=25)
24
25
26  def test_admin_no_puede_crear_admin_central() -> None:
27      with pytest.raises(ValueError):
28          validar_rol_usuario_creacion(rol_creador="ADMIN", rol_nuevo="ADMIN_CENTRAL")
29
30
31  def test_cuenta_bloqueada_falso_si_no_hay_fecha() -> None:
32      assert cuenta_bloqueada(None) is False
33
34
35  def test_calcular_bloqueo_por_intentos() -> None:
36      bloqueo = calcular_bloqueo(5)
37      assert bloqueo is not None
```

```
def validar_dominio_tenant(dominio: str) -> None:
    if not dominio.replace("-", "").isalnum() or dominio != dominio.lower():
        raise ValueError("El dominio solo permite minusculas, numeros y guiones.")
```

```
from datetime import UTC, datetime, timedelta
from uuid import UUID
```

```
from app.core.config import get_settings
```

```
def cuenta_bloqueada(bloqueado_hasta: datetime | None) -> bool:
    if bloqueado_hasta is None:
        return False
    return bloqueado_hasta.replace(tzinfo=UTC) > datetime.now(UTC)
```

```
def calcular_bloqueo(intentos_fallidos: int) -> datetime | None:
    settings = get_settings()
    if intentos_fallidos < settings.max_login_attempts:
        return None
    return datetime.now(UTC) + timedelta(minutes=settings.login_lockout_minutes)
```

```
1 from enum import Enum
```

```
2  
3 class RolUsuario(str, Enum):  
4     ADMIN_CENTRAL = "ADMIN_CENTRAL"  
5     ADMIN = "ADMIN"  
6     DOCENTE = "DOCENTE"  
7     ALUMNO = "ALUMNO"
```

```
8  
9  
10 ROLES_INSTITUCIONALES = {RolUsuario.ADMIN, RolUsuario.DOCENTE, RolUsuario.ALUMNO}
```

```
11  
12  
13 def validar_login_contexto(*, rol: str, tenant_id_presente: bool, dominio_presente: bool) -> None:  
14     if rol == RolUsuario.ADMIN_CENTRAL.value:  
15         if dominio_presente:  
16             raise ValueError("El Administrador Central no requiere dominio de tenant.")  
17         return
```

```
18  
19     if not dominio_presente:  
20         raise ValueError("Debe indicar el dominio del tenant para iniciar sesion.")  
21     if not tenant_id_presente:  
22         raise ValueError("Tenant invalido o inactivo.")
```

```
23  
24 def validar_rol_usuario_creacion(*, rol_creador: str, rol_nuevo: str) -> None:  
25     if rol_creador == RolUsuario.ADMIN_CENTRAL.value:  
26         if rol_nuevo == RolUsuario.ADMIN_CENTRAL.value:  
27             raise ValueError("No puede crear otro Administrador Central desde esta API.")  
28         return
```

```
29  
30     if rol_creador != RolUsuario.ADMIN.value:  
31         raise ValueError("Solo un Administrador puede gestionar usuarios del tenant.")  
32  
33     if rol_nuevo in {RolUsuario.ADMIN_CENTRAL.value, RolUsuario.ADMIN.value}:  
34         raise ValueError("Un Administrador institucional no puede crear usuarios con ese rol.")
```

```
35  
36  
37 def validar_escal_a_evaluacion(*, minima: float, maxima: float, aprobatoria: float) -> None:  
38     if minima > aprobatoria or aprobatoria > maxima:  
39         raise ValueError(  
40             "La escala debe cumplir: nota_minima <= nota_aprobatoria <= nota_maxima."  
41         )
```

TEST DE PRUEBAS UNITARIAS

```
tests/test_domain.py::test_validar_dominio_tenant_ok PASSED [ 14%]  
tests/test_domain.py::test_validar_dominio_tenant_invalido PASSED [ 28%]  
tests/test_domain.py::test_validar_escal_a_evaluacion_ok PASSED [ 42%]  
tests/test_domain.py::test_admin_no_puede_crear_admin_central PASSED [ 71%]  
tests/test_domain.py::test_cuenta_bloqueada_falso_si_no_hay_fecha PASSED [ 85%]  
tests/test_domain.py::test_calcular_bloqueo_por_intentos PASSED [100%]  
  
===== 7 passed in 0.31s =====
```



GESTIÓN DE PERIODOS Y OFERTA ACADÉMICA



Submódulo	Caso / Función de Prueba	Regla de Negocio Evaluada	Descripción de la Verificación
Gestión de Periodos	<u>test_validar_fechas_periodo_ok / _invalido</u>	Coherencia en fechas de vigencia.	Valida que la fecha de inicio del periodo sea estrictamente anterior a la fecha de finalización.
	<u>test_validar_transicion_estado_periodo_ok</u>	Transiciones de estado lineales.	Asegura el avance ordenado del ciclo de vida: CONFIGURACION → MATRICULA → REGISTRO_NOTAS → CERRADO.
	<u>test_validar_transicion_estado_periodo_retroceder</u>	Bloqueo de retroceso de estados.	Impide que el periodo vuelva a un estado anterior (ej. de MATRICULA a CONFIGURACION).
	<u>test_validar_transicion_estado_periodo_salvar</u>	Bloqueo de saltos de estados.	Evita que se salten fases operativas del ciclo de vida (ej. pasar directo de CONFIGURACION a REGISTRO_NOTAS).

```
from datetime import date
from app.models.schemas import PeriodoEstado

def validar_fechas_periodo(fecha_inicio: date, fecha_fin: date) -> None:
    if fecha_inicio >= fecha_fin:
        raise ValueError("La fecha de inicio debe ser anterior a la fecha de fin.")

def validar_transicion_estado_periodo(estado_actual: str, estado_nuevo: str) -> None:
    # CONFIGURACION -> MATRICULA -> REGISTRO_NOTAS -> CERRADO
    secuencia = [
        PeriodoEstado.CONFIGURACION.value,
        PeriodoEstado.MATRICULA.value,
        PeriodoEstado.REGISTRO_NOTAS.value,
        PeriodoEstado.CERRADO.value,
    ]

    if estado_actual not in secuencia:
        raise ValueError(f"Estado actual inválido: {estado_actual}")
    if estado_nuevo not in secuencia:
        raise ValueError(f"Estado nuevo inválido: {estado_nuevo}")

    idx_actual = secuencia.index(estado_actual)
    idx_nuevo = secuencia.index(estado_nuevo)

    if idx_nuevo != idx_actual + 1:
        raise ValueError(
            f"Transición no permitida: de {estado_actual} a {estado_nuevo}. "
            "La secuencia debe ser: CONFIGURACION -> MATRICULA -> REGISTRO_NOTAS -> CERRADO."
        )
```

```
def test_validar_fechas_periodo_ok() -> None:
    fecha_inicio = date(2026, 3, 1)
    fecha_fin = date(2026, 7, 15)
    # No debería lanzar excepciones
    validar_fechas_periodo(fecha_inicio, fecha_fin)

def test_validar_fechas_periodo_invalido() -> None:
    fecha_inicio = date(2026, 7, 15)
    fecha_fin = date(2026, 3, 1)
    with pytest.raises(ValueError, match="La fecha de inicio debe ser anterior a la fecha de fin."):
        validar_fechas_periodo(fecha_inicio, fecha_fin)

def test_validar_transicion_estado_periodo_ok() -> None:
    # CONFIGURACION -> MATRICULA
    validar_transicion_estado_periodo("CONFIGURACION", "MATRICULA")
    # MATRICULA -> REGISTRO_NOTAS
    validar_transicion_estado_periodo("MATRICULA", "REGISTRO_NOTAS")
    # REGISTRO_NOTAS -> CERRADO
    validar_transicion_estado_periodo("REGISTRO_NOTAS", "CERRADO")

def test_validar_transicion_estado_periodo_retroceder() -> None:
    with pytest.raises(ValueError, match="Transición no permitida"):
        validar_transicion_estado_periodo("MATRICULA", "CONFIGURACION")

def test_validar_transicion_estado_periodo_salto() -> None:
    with pytest.raises(ValueError, match="Transición no permitida"):
        validar_transicion_estado_periodo("CONFIGURACION", "REGISTRO_NOTAS")
```

Submódulo	Caso / Función de Prueba	Regla de Negocio Evaluada	Descripción de la Verificación
Oferta Académica	<u>test_validar_prerrequisitos_curso_ok / _invalido</u>	Validación de prerrequisitos de cursos.	Verifica <u>que</u> si un curso tiene prerrequisito de tipo "aprobación de curso", se especifique el curso requerido.
	<u>test_validar_prerrequisitos_creditos_ok / _invalido</u>	Prerrequisito por créditos mínimos.	Asegura que el valor de créditos mínimos sea estrictamente mayor a cero.
	<u>test_validar_suma_pesos_componentes_ok / _excedido</u>	Límite de peso en componentes.	Exige que la suma de los pesos de evaluación de una sección sea exactamente el 100% (falla si supera el límite).
	<u>test_validar_edicion_seccion_ok / _invalida</u>	Modificación de secciones por estado.	Asegura que solo se puedan crear o modificar secciones cuando el periodo esté en estado CONFIGURACION.

```
from uuid import UUID
from decimal import Decimal
from app.models.schemas import PeriodoEstado

def validar_prerrequisitos(
    tipo_prereq: str,
    id_curso_requerido: UUID | None,
    valor_min_creditos: int | None,
) -> None:
    if tipo_prereq == "APROBACION_CURSO":
        if id_curso_requerido is None:
            raise ValueError(
                "Para el prerrequisito de tipo APROBACION_CURSO se debe especificar el curso requerido."
            )
    elif tipo_prereq == "MINIMO_CREDITOS":
        if valor_min_creditos is None or valor_min_creditos <= 0:
            raise ValueError(
                "Para el prerrequisito de tipo MINIMO_CREDITOS se debe especificar un valor mayor a cero."
            )
    else:
        raise ValueError(f"Tipo de prerrequisito desconocido: {tipo_prereq}")

def validar_suma_pesos_componentes(pesos_existentes: list[Decimal], nuevo_peso: Decimal) -> None:
    suma = sum(pesos_existentes) + nuevo_peso
    if suma > Decimal("100.00"):
        raise ValueError(
            f"La suma de pesos supera el 100% permitido. Suma acumulada solicitada: {suma}%"
        )

def validar_edicion_seccion(estado_periodo: str) -> None:
    if estado_periodo != PeriodoEstado.CONFIGURACION.value:
        raise ValueError(
            f"No se pueden crear ni modificar secciones cuando el periodo está en estado {estado_periodo}."
            "Debe estar en CONFIGURACION."
        )
```

```
def test_validar_prerrequisitos_curso_ok() -> None:
    curso_req = uuid4()
    # No debería lanzar excepciones
    validar_prerrequisitos("APROBACION_CURSO", curso_req, None)
```

```
def test_validar_prerrequisitos_curso_invalido() -> None:
    with pytest.raises(ValueError, match="se debe especificar el curso requerido"):
        validar_prerrequisitos("APROBACION_CURSO", None, None)
```

```
def test_validar_prerrequisitos_creditos_ok() -> None:
    # No debería lanzar excepciones
    validar_prerrequisitos("MINIMO_CREDITOS", None, 120)
```

```
def test_validar_prerrequisitos_creditos_invalido() -> None:
    with pytest.raises(ValueError, match="se debe especificar un valor mayor a cero"):
        validar_prerrequisitos("MINIMO_CREDITOS", None, 0)
    with pytest.raises(ValueError, match="se debe especificar un valor mayor a cero"):
        validar_prerrequisitos("MINIMO_CREDITOS", None, None)
```

```
def test_validar_suma_pesos_componentes_ok() -> None:
    existentes = [Decimal("30.00"), Decimal("40.00")]
    # 30 + 40 + 30 = 100 (válido, no excede 100)
    validar_suma_pesos_componentes(existentes, Decimal("30.00"))
```

```
def test_validar_suma_pesos_componentes_excedido() -> None:
    existentes = [Decimal("30.00"), Decimal("40.00")]
    with pytest.raises(ValueError, match="La suma de pesos supera el 100% permitido"):
        # 30 + 40 + 31 = 101 (inválido)
        validar_suma_pesos_componentes(existentes, Decimal("31.00"))
```

```
def test_validar_edicion_seccion_ok() -> None:
    # No debería lanzar excepciones
    validar_edicion_seccion(PeriodoEstado.CONFIGURACION.value)
```

```
def test_validar_edicion_seccion_invalida() -> None:
    with pytest.raises(ValueError, match="No se pueden crear ni modificar secciones"):
        validar_edicion_seccion(PeriodoEstado.MATRICULA.value)
```

```
----- test session starts -----
platform win32 -- Python 3.14.3, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\dair1\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\dair1\Downloads\Proy_SW585_2826_1_Grupo_4\sga\backend
configfile: pytest.ini
plugins: anyio-4.13.0, asyncio-1.4.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None,
asyncio_default_test_loop_scope=function
collecting ... collected 31 items / 6 deselected / 25 selected

tests/test_periodos_oferta.py::test_validar_fechas_periodo_ok PASSED [ 4%]
tests/test_periodos_oferta.py::test_validar_fechas_periodo_invalido PASSED [ 8%]
tests/test_periodos_oferta.py::test_validar_transicion_estado_periodo_ok PASSED [ 12%]
tests/test_periodos_oferta.py::test_validar_transicion_estado_periodo_retroceder PASSED [ 16%]
tests/test_periodos_oferta.py::test_validar_transicion_estado_periodo_salvar PASSED [ 20%]
tests/test_periodos_oferta.py::test_validar_prerrequisitos_curso_ok PASSED [ 24%]
tests/test_periodos_oferta.py::test_validar_prerrequisitos_curso_invalido PASSED [ 28%]
tests/test_periodos_oferta.py::test_validar_prerrequisitos_creditos_ok PASSED [ 32%]
tests/test_periodos_oferta.py::test_validar_prerrequisitos_creditos_invalido PASSED [ 36%]
tests/test_periodos_oferta.py::test_validar_suma_pesos_componentes_ok PASSED [ 40%]
tests/test_periodos_oferta.py::test_validar_suma_pesos_componentes_excedido PASSED [ 44%]
tests/test_periodos_oferta.py::test_validar_edicion_seccion_ok PASSED [ 48%]
tests/test_periodos_oferta.py::test_validar_edicion_seccion_invalida PASSED [ 52%]
tests/test_services.py::test_periodo_crear_success PASSED [ 56%]
tests/test_services.py::test_periodo_crear_invalid_dates PASSED [ 60%]
tests/test_services.py::test_periodo_crear_conflict_name PASSED [ 64%]
tests/test_services.py::test_periodo_transicionar_success PASSED [ 68%]
tests/test_services.py::test_periodo_transicionar_conflict_active PASSED [ 72%]
tests/test_services.py::test_oferta_crear_plan_estudios_success PASSED [ 76%]
tests/test_services.py::test_oferta_crear_plan_estudios_duplicate PASSED [ 80%]
tests/test_services.py::test_oferta_crear_curso_success PASSED [ 84%]
tests/test_services.py::test_oferta_crear_seccion_success PASSED [ 88%]
tests/test_services.py::test_oferta_crear_seccion_invalid_period_state PASSED [ 92%]
tests/test_services.py::test_oferta_crear_componente_evaluation_success PASSED [ 96%]
tests/test_services.py::test_oferta_crear_componente_evaluation_exceeds_weight PASSED [100%]

----- 25 passed, 6 deselected in 0.95s -----
```



GESTIÓN DE MATRICULA



Test	Qué verifica
test_periodo_en_matricula_ok	Permite matricular si el periodo está en estado MATRÍCULA.
test_periodo_en_matricula_invalido.	Rechaza matricular si el periodo no está en MATRÍCULA.
test_seccion_sin_vacantes	Rechaza inscripción cuando no hay vacantes.
test_seccion_cerrada	Rechaza inscripción en sección cerrada.
test_prerrequisito_no_cumplido	Rechaza si el alumno no aprobó el curso requerido.
test_prerrequisito_cumplido	Permite inscripción cuando cumple prerrequisitos.
test_exceso_creditos	Rechaza si supera el límite máximo de créditos.
test_creditos_dentro_limite	Permite inscripción dentro del límite de créditos.
test_matricula_inactiva	Rechaza operaciones sobre matrícula no activa.
test_inscripcion_ya_retirada	Rechaza operaciones sobre inscripción retirada.

```
1  from uuid import uuid4
2
3  import pytest
4
5  from app.domain.matricula import (
6      validar_inscripcion_activa,
7      validar_limite_creditos,
8      validar_matricula_activa,
9      validar_periodo_en_matricula,
10     validar_prerrequisitos_cumplidos,
11     validar_seccion_con_vacantes,
12 )
13
14
15 def test_periodo_en_matricula_ok() -> None:
16     validar_periodo_en_matricula("MATRICULA")
17
18
19 def test_periodo_en_matricula_invalido() -> None:
20     with pytest.raises(ValueError, match="MATRICULA"):
21         validar_periodo_en_matricula("CONFIGURACION")
22
23
24 def test_seccion_sin_vacantes() -> None:
25     with pytest.raises(ValueError, match="vacantes"):
26         validar_seccion_con_vacantes(0, "ABIERTA")
27
28
29 def test_seccion_cerrada() -> None:
30     with pytest.raises(ValueError, match="abierta"):
31         validar_seccion_con_vacantes(5, "CERRADA")
32
33
34 def test_prerrequisito_no_cumplido() -> None:
35     requerido = uuid4()
36     with pytest.raises(ValueError, match="Prerrequisitos no cumplidos"):
37         validar_prerrequisitos_cumplidos([requerido], set())
38
39
40 def test_prerrequisito_cumplido() -> None:
41     requerido = uuid4()
42     validar_prerrequisitos_cumplidos([requerido], {requerido})
43
44
45 def test_exceso_creditos() -> None:
46     with pytest.raises(ValueError, match="limite maximo"):
47         validar_limite_creditos(16, 4, 18)
48
49
```

```
from uuid import UUID
from uuid import UUID

def validar_periodo_en_matricula(estado_periodo: str) -> None:
    if estado_periodo != "MATRICULA":
        raise ValueError(
            f"El periodo academico debe estar en estado MATRICULA. Estado actual: {estado_periodo}."
        )

def validar_matricula_activa(estado_matricula: str) -> None:
    if estado_matricula != "ACTIVA":
        raise ValueError("La matricula no esta activa.")

def validar_seccion_con_vacantes(vacantes_disponibles: int, estado_seccion: str) -> None:
    if estado_seccion != "ABIERTA":
        raise ValueError(f"La seccion no esta abierta. Estado: {estado_seccion}.")
    if vacantes_disponibles <= 0:
        raise ValueError("La seccion no tiene vacantes disponibles.")

def validar_limite_creditos(
    creditos_actuales: int,
    creditos_nuevos: int,
    creditos_maximos: int,
) -> None:
    total = creditos_actuales + creditos_nuevos
    if total > creditos_maximos:
        raise ValueError(
            f"Excede el limite maximo de creditos ({creditos_maximos}). "
            f"Creditos actuales: {creditos_actuales}, creditos del curso: {creditos_nuevos}."
        )

def validar_prerrequisitos_cumplidos(
    requeridos: list[UUID],
    cumplidos: set[UUID],
) -> None:
    faltantes = set(requeridos) - cumplidos
    if faltantes:
        raise ValueError(
            "Prerrequisitos no cumplidos. "
            f"Cursos requeridos pendientes: {sorted(str(c) for c in faltantes)}"
        )

def validar_inscripcion_activa(estado_inscripcion: str) -> None:
    if estado_inscripcion != "ACTIVA":
        raise ValueError("La inscripcion no esta activa.")
```

```
def test_creditos_dentro_limite() -> None:
    validar_limite_creditos(4, 4, 18)
```

```
def test_matricula_inactiva() -> None:
    with pytest.raises(ValueError, match="activa"):
        validar_matricula_activa("RETIRADA")
```

```
def test_inscripcion_ya_retirada() -> None:
    with pytest.raises(ValueError, match="activa"):
        validar_inscripcion_activa("RETIRADA")
```

```
def validar_limite_creditos(
    creditos_actuales: int,
    creditos_nuevos: int,
    creditos_maximos: int,
) -> None:
    total = creditos_actuales + creditos_nuevos
    if total > creditos_maximos:
        raise ValueError(
            f"Excede el limite maximo de creditos ({creditos_maximos}). "
            f"Creditos actuales: {creditos_actuales}, creditos del curso: {creditos_nuevos}."
        )
```

```
def validar_matricula_activa(estado_matricula: str) -> None:
    if estado_matricula != "ACTIVA":
        raise ValueError("La matricula no esta activa.")
```

```
def validar_inscripcion_activa(estado_inscripcion: str) -> None:
    if estado_inscripcion != "ACTIVA":
        raise ValueError("La inscripcion no esta activa.")
```



```
tests/test_matricula.py::test_periodo_en_matricula_ok PASSED [ 10%]
tests/test_matricula.py::test_periodo_en_matricula_invalido PASSED [ 20%]
tests/test_matricula.py::test_seccion_sin_vacantes PASSED [ 30%]
tests/test_matricula.py::test_seccion_cerrada PASSED [ 40%]
tests/test_matricula.py::test_prerrequisito_no_cumplido PASSED [ 50%]
tests/test_matricula.py::test_prerrequisito_cumplido PASSED [ 60%]
tests/test_matricula.py::test_exceso_creditos PASSED [ 70%]
tests/test_matricula.py::test_creditos_dentro_limite PASSED [ 80%]
tests/test_matricula.py::test_matricula_inactiva PASSED [ 90%]
tests/test_matricula.py::test_inscripcion_ya_retirada PASSED [100%]
```

```
===== 10 passed in 0.01s =====
```

```
gabrieleustaquio@MacBook-Air-de-Gabriel backend %
```





GESTION DE CALIFICACIONES



CASOS DE PRUEBA EN PYTHON

```
class TestValidateGradeValue:
    def test_valid_boundary_values(self):
        validate_grade_value(0, 0, 20)      # minimum
        validate_grade_value(20, 0, 20)     # maximum
        validate_grade_value(14.5, 0, 20)   # mid-range

    def test_valid_0_to_100_scale(self):
        validate_grade_value(0, 0, 100)
        validate_grade_value(100, 0, 100)
        validate_grade_value(72.5, 0, 100)

    def test_above_max_raises(self):
        with pytest.raises(GradeDomainError):
            validate_grade_value(21, 0, 20)

    def test_below_min_raises(self):
        with pytest.raises(GradeDomainError):
            validate_grade_value(-0.01, 0, 20)

    def test_exact_max_is_valid(self):
        validate_grade_value(20, 0, 20)     # should not raise
```

Nombre del Test	Entrada (Input)	Comportamiento Esperado	Lógica de Negocio que Valida
test_equal_weights	[Notas: {nota_1: 15, nota_2: 15}, Pesos: {peso_1: 50, peso_2: 50}]	Calcula el promedio ponderado correcto (ej: 15).	Cálculo Ponderado Simple: Verifica el cálculo correcto del promedio ponderado cuando todos los componentes tienen el mismo peso porcentual.
test_three_component_weights	[Notas: {n1: 18, n2: 14, n3: 16}, Pesos: {p1: 20, p2: 40, p3: 40}]	Calcula el promedio ponderado correcto (ej: 16.2).	Cálculo con Múltiples Componentes: Valida la precisión del cálculo con tres o más componentes de evaluación con diferentes porcentajes.
test_weights_not_100_raises	[Pesos: {n1: 40, p2: 30, p3: 30}]	Calcula el promedio ponderado correcto (ej: 16.2).	Validación de Suma de Pesos: Asegura que el sistema rechace configuraciones de evaluación donde la suma de los pesos de los componentes sea menor o mayor al 100%.
test_weights_not_100_raises_v2	[Notas: {n1: 10, n2: 12}, Pesos: {p1: 60, p2: 30}]	Lanza error WeightsTotalError	Validación donde la suma de los componentes sea menor o mayor al 100%.
test_empty_list_returns_zero	[Lista vacía de componentes]	Retorna 0 por defecto.	Manejo de Lista Vacía: Comprueba que la función devuelva un valor por defecto (ej: cero) cuando no hay componentes de evaluación definidos.

```

def test_create_escalavaluacion(self, db_session):
    tenant = Tenant(id_tenant="t-2", nombre_institucional="UNIV 2",
                    dominio="univ2.edu.pe", zona_horaria="America/Lima")
    db_session.add(tenant)
    db_session.flush()

    escala = EscalaEvaluacion(
        id_escalavaluacion="e-1",
        id_tenant="t-2",
        nombre_escalavaluacion="Vigesimal",
        nota_minima=Decimal("0.00"),
        nota_maxima=Decimal("20.00"),
        nota_aprobatoria=Decimal("10.50")
    )
    db_session.add(escala)
    db_session.flush()

    fetched = db_session.query(EscalaEvaluacion).filter_by(id_escalavaluacion="e-1")
    assert fetched is not None
    assert fetched.nota_aprobatoria == Decimal("10.50")
    assert fetched.nota_minima == Decimal("0.00")

```

test_create_escalavaluacion

Comprueba el
almacenamiento correcto de
escalas numéricas de
evaluación y su relación de
clave foránea con el Tenant.



CIERRE ACADÉMICO



CASOS DE PRUEBA EN PYTHON

```
class TestCalcNotaFinal:
    def test_equal_weights(self):
        calificaciones = [
            {"valor_nota": Decimal("15.00"), "peso_relativo": Decimal("50.00")},
            {"valor_nota": Decimal("13.00"), "peso_relativo": Decimal("50.00")},
        ]
        assert calcular_nota_final(calificaciones) == Decimal("14.00")

    def test_three_component_weights(self):
        calificaciones = [
            {"valor_nota": Decimal("12.00"), "peso_relativo": Decimal("30.00")},
            {"valor_nota": Decimal("16.00"), "peso_relativo": Decimal("30.00")},
            {"valor_nota": Decimal("14.00"), "peso_relativo": Decimal("40.00")},
        ]
        # (12*0.30) + (16*0.30) + (14*0.40) = 3.60 + 4.80 + 5.60 = 14.00
        assert calcular_nota_final(calificaciones) == Decimal("14.00")

    def test_weights_not_100_raises(self):
        calificaciones = [
            {"valor_nota": Decimal("15.00"), "peso_relativo": Decimal("40.00")},
            {"valor_nota": Decimal("15.00"), "peso_relativo": Decimal("40.00")},
        ]
        with pytest.raises(CierreDomainError):
            calcular_nota_final(calificaciones)

    def test_empty_list_returns_zero(self):
        assert calcular_nota_final([]) == Decimal("0.00")
```

Nombre del Test	Entrada (Input)	Comportamiento Esperado	Lógica de Negocio que Valida
test_equal_weights	Componentes: - Nota 15 (Peso 50%) - Nota 13 (Peso 50%)	Retorna nota final: 14.00	Poderación Simétrica: Valida que dos evaluaciones con el mismo peso den el promedio aritmético exacto.
test_three_component_weights	Componentes: - Nota 12 (Peso 30%) - Nota 16 (Peso 30%) - Nota 14 (Peso 40%)	Retorna nota final: 14.00	Ponderaciones Decimales Complejas: Asegura la precisión con pesos y notas asimétricas.
test_weights_not_100_raises	Componentes: - Nota 15 (Peso 40%) - Nota 15 (Peso 40%) (Suma: 80%)	Lanza error CierreDomainError .	Regla de Consistencia Fuerte: No se puede calcular una nota final ni cerrar un acta si la configuración de pesos de evaluación no suma exactamente el 100%.
test_empty_list_returns_zero	Lista vacía []	Retorna nota final: 0.00	Prevención de Errores: Evita divisiones entre cero y comportamientos inesperados ante listas vacías.

```
class TestCalcPromediosPonderados:
    def _make_inscriptions(self):
        return [
            {"codigo_curso": "INF01", "creditos": 4, "nota_final": Decimal("15.00"),
             "estado": "APROBADA", "fecha_orden": 1},
            {"codigo_curso": "MAT01", "creditos": 5, "nota_final": Decimal("10.00"),
             "estado": "DESAPROBADA", "fecha_orden": 1},
        ]

    def test_todos_includes_failed(self):
        ins = self._make_inscriptions()
        # (15*4 + 10*5) / 9 = 110/9 ≈ 12.22
        result = calcular_promedio_ponderado(ins, "TODO")
        assert result == Decimal("12.22")

    def test_solo_aprobados(self):
        ins = self._make_inscriptions()
        # Only INF01 (15, 4 credits): 15.00
        assert calcular_promedio_ponderado(ins, "SOLO_APROBADOS") == Decimal("15.00")

    def test_ultimo_keeps_latest_attempt(self):
        ins = [
            {"codigo_curso": "INF01", "creditos": 4, "nota_final": Decimal("15.00"),
             "estado": "APROBADA", "fecha_orden": 1},
            {"codigo_curso": "MAT01", "creditos": 5, "nota_final": Decimal("10.00"),
             "estado": "DESAPROBADA", "fecha_orden": 1},
            # Retake of MAT01 – higher date_order, should replace the failed one
            {"codigo_curso": "MAT01", "creditos": 5, "nota_final": Decimal("16.00"),
             "estado": "APROBADA", "fecha_orden": 2},
        ]
        # INF01(15,4) + MAT01(16,5) → (60+80)/9 = 140/9 ≈ 15.56
        result = calcular_promedio_ponderado(ins, "ULTIMO")
        assert result == Decimal("15.56")

    def test_empty_list_returns_zero(self):
        assert calcular_promedio_ponderado([], "TODO") == Decimal("0.00")

    def test_only_active_entries_excluded(self):
        ins = [
            {"codigo_curso": "INF01", "creditos": 4, "nota_final": None,
             "estado": "ACTIVA", "fecha_orden": 1},
        ]
        assert calcular_promedio_ponderado(ins, "TODO") == Decimal("0.00")
```

Nombre del Test	Entrada (Input)	Comportamiento Esperado	Lógica de Negocio que Valida
test_todos_includes_failed	Cursos cursados: - Aprobado: 15 (4 créd.) - Desaprobado: 10 (5 créd.)	Retorna promedio: 12.22	Regla TODOS: El promedio ponderado incluye todas las notas registradas (aprobadas y desaprobadas) del estudiante.
test_solo_aprobados	Cursos cursados: - Aprobado: 15 (4 créd.) - Desaprobado: 10 (5 créd.)	Retorna promedio: 15.00	Regla SOLO_APROBADOS: Se calculan promedios ignorando los créditos e impacto de materias jaladas (regla de algunas universidades).
test_ultimo_keeps_latest_attempt	Historial: - MAT01 Jalado: 10 (Semestre 1) - MAT01 Aprobado: 16 (Semestre 2)	Retorna promedio: 15.56 (Ponderando solo el 15 y el 16)	Regla ULTIMO (Reintentos): Si un alumno repite un curso, el promedio ponderado se calcula tomando solo la nota del intento más reciente, descartando los anteriores.
test_empty_list_returns_zero	Lista de cursos vacía []	Retorna promedio: 0.00	Casos Iniciales: Manejo correcto de alumnos ingresantes que aún no tienen notas.
test_only_active_entries_excluded	Cursos activos: - Curso sin nota final (None)	Retorna promedio: 0.00	Exclusión de Cursos en Progreso: Asegura que los cursos que el alumno está llevando activamente en el ciclo actual no afecten su promedio histórico.

```
class TestEvaluarPolitica:
    def test_mayor_que(self):
        assert evaluar_politica_condicion(3, 2, "MAYOR_QUE") is True
        assert evaluar_politica_condicion(2, 2, "MAYOR_QUE") is False

    def test_mayor_igual(self):
        assert evaluar_politica_condicion(2, 2, "MAYOR_IGUAL") is True
        assert evaluar_politica_condicion(1, 2, "MAYOR_IGUAL") is False

    def test_igual(self):
        assert evaluar_politica_condicion(5, 5, "IGUAL") is True
        assert evaluar_politica_condicion(4, 5, "IGUAL") is False

    def test_menor_igual(self):
        assert evaluar_politica_condicion(2, 3, "MENOR_IGUAL") is True
        assert evaluar_politica_condicion(4, 3, "MENOR_IGUAL") is False

    def test_menor_que(self):
        assert evaluar_politica_condicion(1, 2, "MENOR_QUE") is True
        assert evaluar_politica_condicion(2, 2, "MENOR_QUE") is False

    def test_invalid_operator_raises(self):
        with pytest.raises(CierreDomainError):
            evaluar_politica_condicion(1, 2, "DISTINTO_DE")
```

Nombre del Test	Entrada / Operación	Comportamiento Esperado	Lógica de Negocio que Valida
test_mayor_que	3 > 2	Retorna True	Valida el operador de comparación estrictamente mayor (>).
test_mayor_igual	2 >= 2	Retorna True	Valida el operador lógico mayor o igual (>=).
test_igual	5 == 5	Retorna True	Valida la igualdad exacta de umbral (=).
test_menor_igual	2 <= 3	Retorna True	Valida el operador lógico menor o igual (<=).
test_menor_que	1 < 2	Retorna True	Valida el operador estrictamente menor (<).
test_invalid_operator_raises	Operador "DISTINTO_DE"	Lanza error CierreDomainError.	Restricción de Operadores: Asegura que el motor de políticas solo ejecute operaciones lógicas permitidas.

PRUEBA PYTEST

```
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-8.3.4, pluggy-1.6.0
rootdir: C:\Users\LUIS
PAUCAR\Documents\multitenant\Proy_SW505_2026_1_Grupo_4\sga\backend
configfile: pytest.ini
plugins: anyio-4.13.0, asyncio-0.24.0
asyncio: mode=Mode.AUTO, default_loop_scope=None
collected 23 items

tests\test_academic_lifecycle.py ..... [100%]

===== 23 passed in 0.41s =====
```



PRUEBAS INTEGRALES



Secuencia / Archivo	Método y Endpoint	Rol Autorizado	Objetivo de Negocio / Qué Valida
01 01 Registrar Calificaciones.bru	POST /calificaciones/secciones/{id_seccion}/componentes/{id_componente}	DOCENTE / ADMINISTRADOR	Ingreso de Notas en Borrador: Permite al docente de la sección registrar una lista de notas por lote para un componente de evaluación específico (ej: Práctica 1) en estado borrador.
02 02 Publicar Componente.bru	PUT /calificaciones/secciones/{id_seccion}/componentes/{id_componente}/publicar	DOCENTE (Coordinador) / ADMINISTRADOR	Publicación de Notas: Cambia el estado del componente de evaluación a PUBLICADO. A partir de este momento, las notas se hacen visibles para los alumnos y se bloquea su edición ordinaria.
03 03 Corregir Calificacion.bru	POST /calificaciones/{id_calificacion}/corregir	ADMINISTRADOR	Corrección Administrativa: Permite modificar una nota publicada. Requiere una justificación formal y el ID de un administrador, registrando un rastro de auditoría inmutable en la tabla correccion_nota.
04 04 Cerrar Acta.bru	POST /cierre/secciones/{id_seccion}/cerrar-acta	DOCENTE (Coordinador) / ADMINISTRADOR	Cierre del Curso: Calcula las notas finales ponderadas de cada alumno inscrito, actualiza el estado de las inscripciones (ej: de ACTIVA a APROBADA o DESAPROBADA) y cierra el registro de la sección.
05 05 Cerrar Periodo.bru	POST /cierre/periodos/{id_periodo}/cerrar	ADMINISTRADOR	Cierre Académico Semestral: Procesa el lote de alumnos matriculados, calcula el Promedio Semestral (PPS) y Acumulado (PPA), genera snapshots de promedios y aplica políticas de condiciones académicas (ej: riesgo por trica).
06 06 Obtener Historial.bru	GET /historial/alumnos/{id_perfil_alumno}	ALUMNO (Propio) / DOCENTE / ADMINISTRADOR	Consulta de Historial JSON: Retorna la información completa consolidada de la historia del alumno agrupada por periodo, promedios semestrales e historial de condiciones de riesgo académico.
07 07 Descargar PDF Record.bru	GET /historial/alumnos/{id_perfil_alumno}/pdf	ALUMNO (Propio) / ADMINISTRADOR	Récord de Notas en PDF: Descarga el documento oficial generado dinámicamente con el récord de notas formateado y sellos digitales de seguridad.

CICLO DE VIDA DEL PERIODO ACADÉMICO

- `test_periodo_integration_flow:`
 - Simula la creación de un periodo académico en estado `CONFIGURACION`.
 - Valida la transición a `MATRICULA`.
 - Verifica la restricción de exclusividad (intento de abrir un segundo periodo simultáneo lanza un error `409 Conflict`).
 - Realiza limpieza transicionando los periodos creados a `REGISTRO_NOTAS` y finalmente a `CERRADO`.

```
61 @pytest.mark.asyncio
62 async def test_periodo_integration_flow(client: AsyncClient) -> None:
63     from app.core.database import db
64     # Limpiar cualquier periodo activo remanente de ejecuciones previas
65     async with db.connection().acquire() as conn:
66         await conn.execute(
67             "UPDATE periodo_academico SET estado = 'CERRADO' WHERE estado IN ('MATRICULA', 'REGISTRO_NOTAS')"
68         )
69
70     headers = await get_admin_headers(client)
71
72     # 1. Crear periodo 1 (max_length=20)
73     periodo1_name = f"P1-{os.getpid()}"
74     res_p1 = await client.post(
75         "/api/v1/periodos",
76         json={
77             "nombre_periodo": periodo1_name,
78             "fecha_inicio": "2026-03-01",
79             "fecha_fin": "2026-07-20",
80         },
81         headers=headers,
82     )
83     assert res_p1.status_code == 201
84     p1 = res_p1.json()
85     assert p1["nombre_periodo"] == periodo1_name
86     assert p1["estado"] == "CONFIGURACION"
87     p1_id = p1["id"]
88
89     # 2. Listar periodos y encontrar el nuevo
90     res_list = await client.get("/api/v1/periodos", headers=headers)
91     assert res_list.status_code == 200
92     periodos = res_list.json()
93     found = any(p["id"] == p1_id for p in periodos)
94     assert found, "El periodo creado no se encontró en la lista."
95
96     # 3. Transicionar periodo 1 a MATRICULA
97     res_trans1 = await client.post(
98         f"/api/v1/periodos/{p1_id}/transicion",
99         json={"estado_nuevo": "MATRICULA"},
100         headers=headers,
101     )
102     assert res_trans1.status_code == 200
103     assert res_trans1.json()["estado"] == "MATRICULA"
```

```
# 4. Crear periodo 2 (max_length=20)
periodo2_name = f"P2-{os.getpid()}"
res_p2 = await client.post(
    "/api/v1/periodos",
    json={
        "nombre_periodo": periodo2_name,
        "fecha_inicio": "2026-08-01",
        "fecha_fin": "2026-12-15",
    },
    headers=headers,
)
assert res_p2.status_code == 201
p2_id = res_p2.json()["id"]

# 5. Intentar transicionar periodo 2 a MATRICULA (Debe fallar con 409 por simultaneidad activa)
res_trans2_fail = await client.post(
    f"/api/v1/periodos/{p2_id}/transicion",
    json={"estado_nuevo": "MATRICULA"},
    headers=headers,
)
assert res_trans2_fail.status_code == 409

# 6. Limpiar estado transicionando P1 a CERRADO
await client.post(
    f"/api/v1/periodos/{p1_id}/transicion",
    json={"estado_nuevo": "REGISTRO_NOTAS"},
    headers=headers,
)
await client.post(
    f"/api/v1/periodos/{p1_id}/transicion",
    json={"estado_nuevo": "CERRADO"},
    headers=headers,
)
```

OFERTA ACADÉMICA Y COMPONENTES DE EVALUACIÓN

1. Configuración Central (`ADMIN_CENTRAL`): El Administrador Central crea la Escala de Calificaciones (ej. *Escala Vigesimal 0-20*) y los Componentes de Evaluación permitidos (ej. *Práctica Calificada - PC*) para el Tenant de prueba.

2. Diseño Curricular (`ADMIN` local):

- Crea un Plan de Estudios y lo activa.
- Crea un Curso (ej. *Ingeniería de Requerimientos*, con 4 créditos).
- Asocia el Curso al Plan de Estudios.

3. Planificación de Secciones (Periodo en `CONFIGURACION`):

- Crea la Sección A para dicho curso en el periodo recién abierto. Como el periodo está en `CONFIGURACION`, la base de datos lo permite exitosamente (`201 Created`).

4. Regla de Negocio 1: Suma de Pesos de Evaluación:

- Intenta agregar componentes de evaluación. Agrega una PC1 con peso del 40%\$.
- Intenta agregar una PC2 con peso del 70%\$ (Suma acumulada = 110%\$). El sistema rechaza la operación con un error `400 Bad Request` porque la suma de pesos no puede exceder el 100%\$ permitido en la sección.

5. Regla de Negocio 2: Cierre de Configuración de Secciones:

- El periodo transiciona al estado `MATRICULA`.
- Intenta crear una nueva Sección B para el curso. El sistema deniega la operación con un error `400 Bad Request`, validando que no se pueden crear ni modificar secciones una vez que se inicia el periodo de matrícula.

```

@pytest.mark.asyncio
async def test_oferta_academica_integration_flow(client: AsyncClient) -> None:
    central_headers = await get_admin_central_headers(client)
    admin_headers = await get_admin_headers(client)
    tenant_id = "a1111111-1111-1111-1111-111111111111"

    # 1. Obtener o crear escala de evaluación
    res_escalas = await client.get(
        f"/api/v1/tenants/{tenant_id}/catalogos/escalas-evaluacion",
        headers=central_headers,
    )
    assert res_escalas.status_code == 200
    escalas = res_escalas.json()
    if not escalas:
        res_scale_create = await client.post(
            f"/api/v1/tenants/{tenant_id}/catalogos/escalas-evaluacion",
            json={
                "nombre_escalas": "Vigesimal",
                "nota_minima": 0.0,
                "nota_maxima": 20.0,
                "nota_aprobatoria": 10.5,
            },
            headers=central_headers,
        )
        assert res_scale_create.status_code == 201
        escala_id = res_scale_create.json()["id"]
    else:
        escala_id = escalas[0]["id"]

    # 2. Obtener o crear tipo de componente "PC"
    res_tipos = await client.get(
        f"/api/v1/tenants/{tenant_id}/catalogos/tipos-componente",
        headers=central_headers,
    )
    assert res_tipos.status_code == 200
    tipos = res_tipos.json()
    pc_tipo = next((t for t in tipos if t["codigo"] == "PC"), None)
    if not pc_tipo:
        res_tipo_create = await client.post(
            f"/api/v1/tenants/{tenant_id}/catalogos/tipos-componente",
            json={
                "codigo": "PC",
                "nombre": "Practica Calificada",
                "descripcion": "Evaluacion de practica continua",
            },
            headers=central_headers,
        )
        assert res_tipo_create.status_code == 201
        tipo_comp_id = res_tipo_create.json()["id"]
    else:
        tipo_comp_id = pc_tipo["id"]

```

```

# 3. Crear Periodo Académico
period_name = f"P-OF-{os.getpid()}"
if len(period_name) > 20:
    period_name = period_name[:20]
res_period = await client.post(
    "/api/v1/periodos",
    json={
        "nombre_periodo": period_name,
        "fecha_inicio": "2026-03-01",
        "fecha_fin": "2026-07-20",
    },
    headers=admin_headers,
)
assert res_period.status_code == 201
period_id = res_period.json()["id"]

# 4. Crear Plan de Estudios
carrera_name = f"Carrera-{os.getpid()}"
res_plan = await client.post(
    "/api/v1/oferta/planes-estudio",
    json={
        "carrera": carrera_name,
        "version_plan": "2026-I",
        "creditos_totales": 200,
    },
    headers=admin_headers,
)
assert res_plan.status_code == 201
plan_id = res_plan.json()["id"]

# 5. Activar Plan de Estudios
res_act_plan = await client.put(
    f"/api/v1/oferta/planes-estudio/{plan_id}/activar",
    headers=admin_headers,
)
assert res_act_plan.status_code == 200
assert res_act_plan.json()["estado"] == "ACTIVO"

```

```

# 6. Crear Curso
course_code = f"C-{os.getpid()}"
if len(course_code) > 20:
    course_code = course_code[:20]
res_course = await client.post(
    "/api/v1/oferta/cursos",
    json={
        "codigo_curso": course_code,
        "nombre_curso": "Ingenieria de Requerimientos",
        "creditos": 4,
        "tipo_curso": "OBLIGATORIO",
        "ciclo_sugerido": 5,
    },
    headers=admin_headers,
)
assert res_course.status_code == 201
course_id = res_course.json()["id"]

# 7. Asociar Curso al Plan de Estudios
res_assoc = await client.post(
    f"/api/v1/oferta/planes-estudio/{plan_id}/cursos",
    json={
        "id_curso": course_id,
        "ciclo_en_plan": 5,
        "es_obligatorio": True,
    },
    headers=admin_headers,
)
assert res_assoc.status_code == 201

# 8. Crear Sección (Periodo en CONFIGURACION, OK)
res_sec = await client.post(
    "/api/v1/oferta/secciones",
    json={
        "id_periodo": period_id,
        "id_curso": course_id,
        "codigo_seccion": "A",
        "vacantes_maximas": 40,
    },
    headers=admin_headers,
)
assert res_sec.status_code == 201
seccion_id = res_sec.json()["id"]

```

```

# 9. Crear Componente de Evaluación (40%, OK)
res_comp1 = await client.post(
    f"/api/v1/oferta/secciones/{seccion_id}/componentes",
    json={
        "id_tipo_componente": tipo_comp_id,
        "id_escalas": escala_id,
        "peso_relativo": 40.0,
        "orden_presentacion": 1,
    },
    headers=admin_headers,
)
assert res_comp1.status_code == 201

# 10. Crear Componente de Evaluación Excesivo (70%, Error 400)
res_comp2 = await client.post(
    f"/api/v1/oferta/secciones/{seccion_id}/componentes",
    json={
        "id_tipo_componente": tipo_comp_id,
        "id_escalas": escala_id,
        "peso_relativo": 70.0,
        "orden_presentacion": 2,
    },
    headers=admin_headers,
)
assert res_comp2.status_code == 400

# 11. Transicionar Periodo a MATRICULA
res_trans = await client.post(
    f"/api/v1/periodos/{period_id}/transicion",
    json={"estado_nuevo": "MATRICULA"},
    headers=admin_headers,
)
assert res_trans.status_code == 200
assert res_trans.json()["estado"] == "MATRICULA"

# 12. Intentar crear sección (Periodo en MATRICULA, Error 400)
res_sec_fail = await client.post(
    "/api/v1/oferta/secciones",
    json={
        "id_periodo": period_id,
        "id_curso": course_id,
        "codigo_seccion": "B",
        "vacantes_maximas": 40,
    },
    headers=admin_headers,
)
assert res_sec_fail.status_code == 400

```

```

# 13. Limpiar estado transicionando el periodo creado a CERRADO
await client.post(
    f"/api/v1/periodos/{period_id}/transicion",
    json={"estado_nuevo": "REGISTRO_NOTAS"},
    headers=admin_headers,
)
await client.post(
    f"/api/v1/periodos/{period_id}/transicion",
    json={"estado_nuevo": "CERRADO"},
    headers=admin_headers,
)

```



```
----- test session starts -----  
platform win32 -- Python 3.14.3, pytest-9.0.3, pluggy-1.6.0  
cachedir: .pytest_cache  
rootdir: C:\Users\dair1\Downloads\Proy_SM585_2026_1_Grupo_4\sga\backend  
configfile: pytest.ini  
plugins: anyio-4.13.0, asyncio-1.4.0  
asyncio: mode=Mode.AUTO  
collecting ... collected 4 items / 2 deselected / 2 selected  
  
tests/test_integration.py::test_periodo_integration_flow PASSED  
tests/test_integration.py::test_oferta_academica_integration_flow PASSED  
  
----- 2 passed, 2 deselected in 36.99s -----
```



**MUCHAS
GRACIAS**

